# C# Program Lifecycle & Execution Model

## 1. What is Program Lifecycle?

Program Lifecycle describes the journey of the application:

- Writing the source code
- Compiling the code
- Generating output files
- Running the application on a target environment (OS)

---

## 2. Traditional Compilation (C / C++)

In traditional languages like C or C++:

1. Write source code
2. Compile code
3. Generate **Machine Code**
4. Produce a native `.exe`
5. Run directly on the OS

### Limitation:

- Each `.exe` is tied to a specific Operating System
- Windows executable ≠ Linux executable ≠ macOS executable
- Multiple OS → multiple compilations

---

## 3. Target Environment

Target Environment refers to the operating system where the application will run:

- Windows
- Linux
- macOS

Each environment requires its own compatible executable.

---

# 4. The Main Problem

Is it possible to:

> Write the application **once** and run it on multiple operating systems?

- Hard with C / C++
- Solved using **Intermediate Languages** (Java & C#)

---

# 5. C# Compilation Model

When writing C# code:

- Source file: `.cs`
- Compilation output:
  - ❌ Not native machine code
  - ❌ Not OS-specific
  - ✅ Produces **MSIL**

## MSIL (Microsoft Intermediate Language)

- Not machine language
- Not C#
- Platform-independent intermediate code

> Similar to Java Bytecode

---

# 6. Is the C# `.exe` Really Executable?

Yes, but:

- It does **not** contain machine code
- It contains **Intermediate Language (MSIL)**

To run it, an additional runtime is required.

---

# 7. Runtime & Execution

C# uses a runtime environment called:

### CLR (Common Language Runtime)

Execution flow:
MSIL → CLR → Machine Code → Run

- Conversion happens **at runtime**
- Known as **JIT (Just-In-Time Compilation)**

---

## 8. Responsibility Breakdown

### Developer:

- Writes the code
- Compiles the application **once**

### Client:

- Runs the application
- Must have:
  - .NET Runtime installed

---

## 9. Performance Comparison

### Why is C faster than C#?

- C → Direct machine code
- C# → Runtime + JIT translation

> Performance difference is usually minor and unnoticeable to users.

---

## 10. Key Advantage of C#

### Write Once – Run Anywhere (With Runtime)

- Same MSIL code

- Different CLR implementations per OS:
  - Windows CLR
  - Linux CLR
  - macOS CLR

---

## 11. Final Mental Model

C# Source Code
↓
MSIL (Intermediate Language)
↓
CLR (Based on OS)
↓
Native Machine Code
↓
Program Execution

---

## Summary

- C# does not compile directly to machine code
- Uses MSIL as an intermediate step
- CLR converts MSIL to native code at runtime
- Enables cross-platform execution

# Cross Platform vs Native & .NET Framework Internals

---

## 1. Difference Between Cross Platform and Native

### Native Applications

- Built specifically for **one OS**
- Uses all OS features directly
- Highest performance
- Strong access to hardware & system APIs

Examples:

- Swift for iOS
- Kotlin/Java for Android
- C/C++ native apps

---

### Cross Platform Applications

- Write code once
- Run on multiple operating systems
- Depends on an intermediate layer or runtime
- Limited access to some OS-specific features

Examples:

- Flutter
- .NET (Core / Modern .NET)
- Java

---

## 2. Native vs Cross Platform (Key Comparison)

| Aspect | Native | Cross Platform |
| --- | --- | --- |

| Aspect | Native | Cross Platform |
|---|---|---|
| Performance | High | Slightly lower |
| OS Features | Full access | Partial access |
| Code Reuse | Low | High |
| Development Speed | Slower | Faster |
| Platform Dependency | High | Low |

---

# 3. Flutter vs Swift Example

## Swift (Native)

- Targets Apple OS directly
- Full access to iOS capabilities
- Best performance on iOS
- Platform-specific

## Flutter (Cross Platform)

- One codebase
- Runs on:
  - iOS
  - Android
  - Web
- Uses abstraction layer
- Some OS features may be limited

---

# 4. .NET Platform Overview

.NET Platform is a **collection of components**, not a single tool.

Main components:

- .NET Framework
- CLR
- Libraries
- Tools (Visual Studio)

---

# 5. Components of .NET Framework

## 1️⃣ CLR – Common Language Runtime

Responsible for:

- Running the application
- Converting IL to native machine code
- Memory management
- Security
- Exception handling

Execution flow:

C# Code → Compile → IL → CLR → Native Code

---

# 6. Intermediate Language (IL)

- IL is generated after compilation
- Not machine code
- Not C#
- Platform independent

## Important Concept:

IL can be **reverse engineered**

- IL → C#
- This is why decompilers exist

---

# 7. Multi-Language Support in .NET

.NET allows multiple languages to compile into the **same IL**:

- C#
- VB.NET
- F#
- J#

All languages:

Language → IL → CLR → Execution

---

# 8. Project Types & Output

When creating a project, you choose the output type:

| Project Type | Output |
|---|---|
| Console App | `.exe` |
| Windows Forms | `.exe` |
| Web App | Hosted (API / MVC) |
| Class Library | `.dll` |

---

# 9. DLL – Dynamic Linking Library

- DLL contains **IL code**
- Not executed alone
- Used by applications at runtime
- Shared between multiple apps

---

# 10. Linking Concept

## In C/C++

- Code + libraries merged into exe
- Larger exe size
- Library code duplicated per app

## In .NET

- Libraries are shared
- Linked at **runtime**
- CLR handles linking

---

# 11. Runtime Linking in .NET

- Compiler stops after producing IL
- At runtime:
  - CLR loads required libraries
  - Links them dynamically
  - Executes code

This is called:

### Dynamic Linking

---

## 12. Benefits of Dynamic Linking

1. Smaller executable size
2. Shared memory usage
3. Faster updates
4. Updating a DLL does NOT affect all apps
5. Efficient memory usage

---

## 13. Base Class Library (BCL)

- Core library of .NET
- Contains ready-to-use classes
- Examples:
  - Collections
  - File handling
  - Networking
  - Threading

BCL is part of:

.NET Framework

---

## 14. Using Custom Libraries

When you create your own library:

- It is NOT part of .NET Framework
- You must ship it with your app

Options:

1. Embed library during compile
2. Link library dynamically at runtime

---

# 15. Runtime Library Resolution

At runtime:

- CLR detects required library
- Searches for it:
  - In .NET Framework
  - Or in application directory
- Loads and executes it

---

# 16. Developer vs Client Environment

## Developer:

- Has:
  - .NET Framework
  - Visual Studio
  - Libraries

## Client:

- Only needs:
  - .NET Runtime
  - Application exe + custom libraries

---

# 17. Final Mental Model

Source Code
↓
Compile
↓
IL
↓

CLR

↓

Load Libraries

↓

Native Code

↓

Execution

---

# Summary

- Native apps are faster and OS-specific
- Cross platform apps trade performance for flexibility
- .NET uses IL + CLR for execution
- Libraries are linked dynamically at runtime
- BCL provides core reusable functionality